

Method

Introduction to Methods

- Reusable blocks of code
- Perform specific tasks
- Can be called from any part of the code
- Consists of input parameters and return values
- Enhances code reusability, modularity, and maintainability
- Also known as function in some other languages

Types of Methods

1. Pre-defined/ built-in method:

These methods are already defined in java libraries e.g: `.length()`, `.equals()`, `.compareTo()`, `.toLowerCase()` etc.

1. User-defined method:

These are the methods written by the programmer.

In this lesson, we will learn how to write user-defined methods.

Static Method

- A static method belongs to the class and not to the instance of a class.
- As it belongs to the class, it can be called without creating any object.
- We use the keyword “static” to define a method as static.
- We can call a static method from inside any other static method.

User-Defined Methods

There are mainly 2 parts of using a user-defined method:

1. Method declaration
2. Method calling

Method Declaration can be further broken down into more parts.

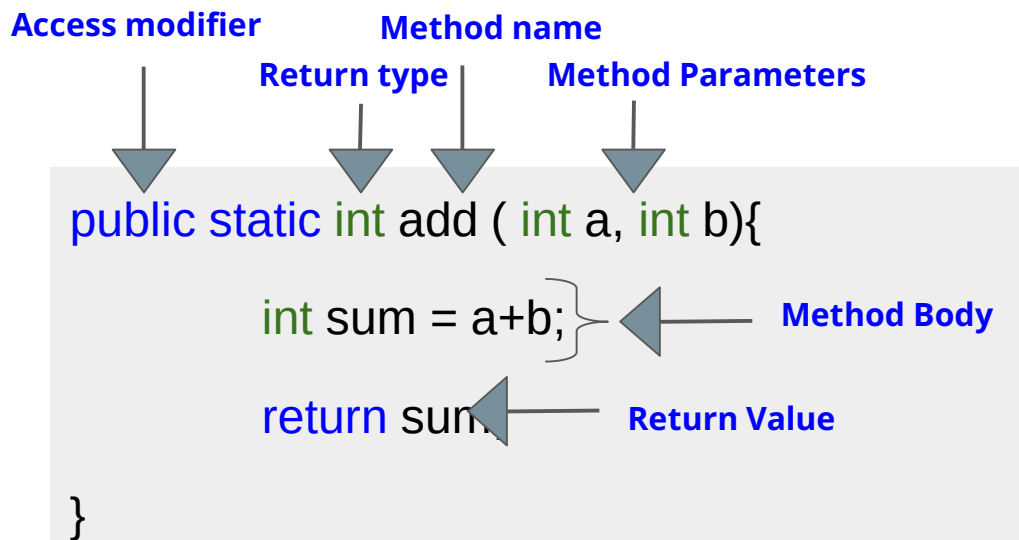
Method Declaration

- A method in java is written outside the main() method but inside the class.
- The following convention is followed when declaring a method:

```
access_modifier static return_type method_name (parameter_type parameter_name) {  
    method body;  
    return return_value;  
}
```

Method Declaration Cont.

Declaring a method that adds two numbers received as parameters.



Method Body

- Any set of code lines written inside the method.
- Any variable declared inside the scope of a method is not accessible from outside including the parameter variables.
- No line is executed after the “return” statement.

Return Type

- To declare a method we need to define what type of variable the method is returning.
- If we do not explicitly return any data or variable from the method it returns Null and that is declared as “void” in the method declaration.

1 void return

```
static void print_sum( int a, int b) {  
    int sum = a+b;  
    System.out.println(sum);  
} //this method does not return any  
value.
```

2 int return

```
static int sum( int a, int b) {  
    int sum = a+b;  
    return sum;  
} //this method returns an integer type  
value.
```

3 boolean return

```
static boolean check( int a) {  
    if(a>10){  
        return true;  
    }  
    else{  
        return false;  
    }  
} //this method returns a boolean type  
value.
```

4 String return

```
static String okay(int a, String str1) {  
    return (a+2)+str1;  
} //this method returns a String type  
value.
```

5 double return

```
static double point(int a) {  
    double db = a/10.0;  
    return db;  
} //this method returns a double type  
variable.
```

Method Call

- To run a method we MUST call the method.
- Method call is done inside the main() method.
- The number of arguments passed must be equal to parameters received.
- The sequence of the parameters passed is important.
- The type of the arguments passed must be same as type of parameters being received.
- After method call, a value is returned to the line of method call.

Method Call Cont.

Calling the previously written add method.

Receiving the
value of sum

```
public static void main(String [] args){  
    int answer = add(4,5);  
    System.out.println(answer);  
}
```

Passing the same
type of arguments as
declared (int)

Welcome to DrJava. Working directory is E:\Java

> run function1

9

>

Method Arguments & Parameters

The sequence of the arguments passed corresponds to the variables receiving the parameters.

Method to subtract two numbers

```
static void subtract( int a, int b) {  
    int diff = a-b;  
    System.out.println(diff);  
} //this method does not return any  
value.
```

Calling subtract

```
subtract(2, 3);  
  
subtract(3,2);
```

Output

```
Welcome to DrJava. Working directory is  
E:\Java  
  
> run function1  
  
-1  
  
1  
  
>
```

Here in case 1: subtract(2,3), the values 2 and 3 will be received as int a = 2 and int b = 3. Therefore, the difference is $(2-3) = -1$.

In case 2: int a = 3 and int b = 2 and so difference is $(3-2) = 1$.

Method Signature

- A method is identified based on its name, parameter list it receives and return type.
- The number of arguments passed in method call must be equal to the number of parameters received during method declaration.
- Multiple methods can have same name but different sets of parameters.
- It is not possible to have multiple methods with the same name, same parameters but different return types.

Method Example 1

Writing a method that prints "Hello!"

```
public class func1{  
    public static void sayHello(){  
        System.out.println("Hello!");  
    }  
    public static void main(String []args){  
        sayHello();  
    }  
}
```

No arguments and
no parameters
received

Void return type
as no value is
being returned
from inside

Output

```
Welcome to DrJava. Working directory is  
E:\Java  
> run func1  
Hello!  
>
```

Method Example 2

Writing a method that checks if a number divisible by three or not.

```
public class func1{  
    public static String divThree(int num){  
        if(num%3==0){  
            return "Divisible by 3";  
        }  
        else{  
            return "Not divisible by 3";  
        }  
    }  
    public static void main(String []args){  
        String answer = divThree(13);  
        System.out.println(answer);  
    }  
}
```

This value will be
returned to the line of
method call

13 will be received by
num. And num = 13
will be accessible
inside the function.

This line will invoke a
method call.

Output

Welcome to DrJava. Working directory is
E:\Java

> run func1

Not divisible by 3

>

Method Example 3

Writing a method that prints the largest number out of two numbers.

```
public class func1{  
    public static void findLargest(int num1, int num2){  
        if(num1>num2){  
            System.out.println(num1);  
        }  
        else{  
            System.out.println(num2);  
        }  
    }  
    public static void main(String []args){  
        findLargest(13, -10);  
    }  
}
```

Output

Welcome to DrJava. Working directory is
E:\Java

> run func1

13

>

void return
since no return
statement.

To do

Write a method named `evenPositive` to find even and positive numbers from an array of numbers.

Calling a Method from another Method

- We can call a method from inside another method.
1. method1() called from the main() method.
 2. method2() called from method1().

```
public class Methods {  
    public static void main(String[] args) {  
        method1();  
    }  
    static void method1(){  
        System.out.print("CSE");  
        method2();  
    }  
    static void method2(){  
        System.out.print("110");  
    }  
}
```

```
Welcome to DrJava. Working directory is  
C:\Users\Desktop  
CSE110
```

To Do

Now, can you print a triangular palindrome like the following using a method called `show_triangle()`? Note that you have to take the help of `show_spaces()` and `show_palindrome()` method.

```
    1
   121
  12321
 1234321
123454321
```

- `show_spaces()` method prints a number of spaces passed to its parameter
 - `show_palindrome()` prints a palindrome.
- These methods are already done for you.

```
public class MethodCalling {
    public static void main(String[] args) {
        show_triangle(4);
        System.out.println();
        show_triangle(5);
    }
    static void show_spaces(int a){
        for (int i = 0; i<a; i++){
            System.out.print(" ");
        }
    }
    static void show_palindrome(int b) {
        for (int i = 1; i <= b; i++) {
            System.out.print(i);

            for (int i = b - 1; i >= 1; i--)
            {
                System.out.print(i);
            }
        }

        static void show_triangle(int a){
            //To Do
        }
    }
}
```